



Deep and Shallow Copy in JavaScript

Created by: Neeraj | [LinkedIn: neeraj-kumar1904](#) | [X: @_19_neeraj](#) | [GitHub: Neeraj05042001](#) |

Introduction

Welcome to the world of copying in JavaScript! As you build more complex applications with the MERN stack, you'll frequently work with objects and arrays. Understanding how JavaScript handles copying data is crucial for writing bug-free code and avoiding unexpected behavior.

Imagine you're building a React application where you need to update user profiles , manage shopping cart items , or handle form data . Without proper understanding of copying, you might accidentally modify original data when you only wanted to create a copy. This can lead to confusing bugs where changing one thing unexpectedly affects something else entirely.

In this comprehensive guide, we'll explore everything about deep and shallow copying, starting from the very basics and progressing to advanced techniques used in professional development.



Table of Contents

1. [Understanding JavaScript Data Types](#)
2. [What is Copying?](#)
3. [Primitive vs Reference Types](#)
4. [Shallow Copy Explained](#)
5. [Deep Copy Explained](#)
6. [Methods for Shallow Copying](#)
7. [Methods for Deep Copying](#)
8. [Common Pitfalls and Gotchas](#)
9. [Real-World Examples](#)
10. [Best Practices](#)
11. [Advanced Concepts](#)
12. [Practice Exercises](#)

1. Understanding JavaScript Data Types

Before diving into copying, let's understand JavaScript's data types:

Primitive Types

- number (1, 3.14, -5) 
- string ("hello", 'world') 
- boolean (true, false)  
- undefined 
- null 
- symbol (ES6+) 
- bigint (ES2020+)  

Reference Types

- object ({}, {name: "John"}) 
- array ([], [1, 2, 3]) 
- function (function() {}) 
- date (new Date()) 

 **Key Point:** Primitive types store actual values, while reference types store references (addresses) to values in memory. 

```
//  Primitive types
let a = 5;
let b = a; // b gets the VALUE 5
a = 10;
console.log(a); // 10
console.log(b); // 5 (unchanged) 

//  Reference types
let obj1 = { name: "John" };
let obj2 = obj1; // obj2 gets the REFERENCE to the same object
obj1.name = "Jane";
console.log(obj1.name); // "Jane"
console.log(obj2.name); // "Jane" (changed!) 
```

2. What is Copying?

Copying means creating a duplicate of data.    However, in JavaScript, there are two types of copying:

Assignment (Not Really Copying)

```
let original = { name: "John", age: 25 };
let copy = original; // This is assignment, not copying! ❌

copy.name = "Jane";
console.log(original.name); // "Jane" - Original changed! 😱
```

✅ Actual Copying

Creating a new, independent copy where changes to one don't affect the other. 🔄

3. ⚖️ Primitive vs Reference Types

🧱 Primitive Types (Always Deep Copied)

```
let num1 = 42;
let num2 = num1; // Creates a new copy ✨
num1 = 50;
console.log(num1); // 50
console.log(num2); // 42 (unchanged) ✅

let str1 = "Hello";
let str2 = str1; // Creates a new copy ✨
str1 = "World";
console.log(str1); // "World"
console.log(str2); // "Hello" (unchanged) ✅
```

📦 Reference Types (Need Special Handling)

```
let arr1 = [1, 2, 3];
let arr2 = arr1; // Copies reference, not the array ⚠️
arr1.push(4);
console.log(arr1); // [1, 2, 3, 4]
console.log(arr2); // [1, 2, 3, 4] (also changed!) 😱

let obj1 = { name: "John" };
let obj2 = obj1; // Copies reference, not the object ⚠️
obj1.age = 25;
console.log(obj1); // { name: "John", age: 25 }
console.log(obj2); // { name: "John", age: 25 } (also changed!) 😱
```

4. 🔍 Shallow Copy Explained

A **shallow copy** creates a new object/array    , but nested objects/arrays are still referenced, not copied. 

Visual Representation

```
Original: { a: 1, b: { c: 2 } }  
          ↓  
Shallow: { a: 1, b: →[same reference] }
```

Example

```
let original = {  
  name: "John",  
  hobbies: ["reading", "swimming"],  
  address: {  
    city: "New York",  
    zip: "10001"  
  }  
};  
  
// 🔍 Shallow copy  
let shallowCopy = { ...original };  
  
// ✅ Top-level changes don't affect original  
shallowCopy.name = "Jane";  
console.log(original.name); // "John" (unchanged) ✅  
console.log(shallowCopy.name); // "Jane"  
  
// ⚠️ But nested changes DO affect original  
shallowCopy.hobbies.push("coding");  
console.log(original.hobbies); // ["reading", "swimming", "coding"] (changed!) 🧑  
console.log(shallowCopy.hobbies); // ["reading", "swimming", "coding"]  
  
shallowCopy.address.city = "Boston";  
console.log(original.address.city); // "Boston" (changed!) 🧑  
console.log(shallowCopy.address.city); // "Boston"
```

When to Use Shallow Copy

- ✅ When you only need to modify top-level properties
- ✅ When nested objects/arrays won't be modified
- ⚡ For performance reasons (faster than deep copy)
- ⚙️ In React when updating state with immutable patterns

5. 🐼 Deep Copy Explained

A **deep copy** creates a new object/array and recursively copies all nested objects/arrays. 📦 ➡ 📦 ↺

🎨 Visual Representation

Original: { a: 1, b: { c: 2 } }

↓

Deep: { a: 1, b: { c: 2 } } [completely independent] 📦

💡 Example

```
let original = {
  name: "John",
  hobbies: ["reading", "swimming"],
  address: {
    city: "New York",
    zip: "10001"
  }
};

// 🐼 Deep copy (we'll see different methods later)
let deepCopy = JSON.parse(JSON.stringify(original));

// ✅ No changes affect the original
deepCopy.name = "Jane";
deepCopy.hobbies.push("coding");
deepCopy.address.city = "Boston";

console.log(original.name); // "John" (unchanged) ✅
console.log(original.hobbies); // ["reading", "swimming"] (unchanged) ✅
console.log(original.address.city); // "New York" (unchanged) ✅

console.log(deepCopy.name); // "Jane"
console.log(deepCopy.hobbies); // ["reading", "swimming", "coding"]
console.log(deepCopy.address.city); // "Boston"
```

🎯 When to Use Deep Copy

- ✅ When you need complete independence from original
- ✅ When nested objects/arrays will be modified
- ✅ When working with complex data structures
- 🔄 In Redux for state management

6. 🛠️ Methods for Shallow Copying

6.1 🌟 Spread Operator (...) - Recommended

```
// 📄 Objects
let original = { a: 1, b: 2, c: { d: 3 } };
let copy = { ...original };

// 📊 Arrays
let originalArray = [1, 2, [3, 4]];
let copyArray = [...originalArray];
```

6.2 🛠️ Object.assign()

```
// 📄 Objects only
let original = { a: 1, b: 2 };
let copy = Object.assign({}, original);

// 🔄 You can also merge objects
let copy2 = Object.assign({}, original, { c: 3 });
```

6.3 📊 Array Methods

```
let original = [1, 2, 3];

// ✂️ slice() - creates shallow copy
let copy1 = original.slice();

// 📦 Array.from()
let copy2 = Array.from(original);

// 🔗 concat()
let copy3 = [].concat(original);
```

6.4 🎯 Object.create() + Object.getPrototypeOf()

```
// 🛠️ For objects with getters/setters
let original = { a: 1, b: 2 };
let copy = Object.create(
  Object.getPrototypeOf(original),
  Object.getPrototypeOf(original)
);
```

💡 Practical Example: React State Update

```
// 🧠 In a React component
const [user, setUser] = useState({
  name: "John",
  preferences: { theme: "dark", language: "en" }
});

// ✅ Correct way to update nested state
const updateTheme = (newTheme) => {
  setUser({
    ...user, // shallow copy user
    preferences: {
      ...user.preferences, // shallow copy preferences
      theme: newTheme
    }
  });
};
```

7. 🛠️ Methods for Deep Copying

7.1 📋 JSON.parse(JSON.stringify()) - Simple but Limited

```
let original = {
  name: "John",
  age: 25,
  hobbies: ["reading", "swimming"],
  address: { city: "NY", zip: "10001" }
};

let deepCopy = JSON.parse(JSON.stringify(original));
```

⚠️ Limitations:

- ❌ Doesn't handle functions
- ❌ Doesn't handle undefined values
- ❌ Doesn't handle Symbols
- ❌ Doesn't handle Dates properly
- ❌ Doesn't handle circular references

```
// 🚨 Problems with JSON method
let problematic = {
  func: function() { return "hello"; },
  undef: undefined,
  date: new Date(),
  symbol: Symbol("test")
};
```

```
};
```

```
let copy = JSON.parse(JSON.stringify(problematic));
console.log(copy);
// Output: { date: "2023-..." } - func, undef, symbol are lost! 🤖
```

7.2 🛠 Custom Recursive Function

```
function deepCopy(obj) {
  // 🚫 Handle null or undefined
  if (obj === null || typeof obj !== "object") {
    return obj;
  }

  // 📅 Handle Date
  if (obj instanceof Date) {
    return new Date(obj.getTime());
  }

  // 📦 Handle Array
  if (Array.isArray(obj)) {
    return obj.map(item => deepCopy(item));
  }

  // 📄 Handle Object
  const copiedObj = {};
  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      copiedObj[key] = deepCopy(obj[key]);
    }
  }

  return copiedObj;
}

// 🎯 Usage
let original = {
  name: "John",
  birthDate: new Date("1990-01-01"),
  hobbies: ["reading", { type: "sports", list: ["swimming", "running"] }]
};

let copy = deepCopy(original);
```

7.3 📦 Using Lodash Library

```
// 📦 First install: npm install lodash
const _ = require('lodash');
```



```
let original = { /* complex object */ };
let deepCopy = _.cloneDeep(original);

// ⭐ Lodash handles most edge cases correctly
```

7.4 Using structuredClone() - Modern Approach

```
// ⭐ Available in modern browsers and Node.js 17+
let original = {
  name: "John",
  date: new Date(),
  nested: { array: [1, 2, 3] }
};

let deepCopy = structuredClone(original);
```

8. ⚠ Common Pitfalls and Gotchas

8.1 🏠 The "Nested Object Trap"

```
// ❌ Common mistake
let users = [
  { name: "John", settings: { theme: "dark" } },
  { name: "Jane", settings: { theme: "light" } }
];

let usersCopy = [...users]; // Shallow copy of array
usersCopy[0].settings.theme = "blue"; // Modifies original! 😱

console.log(users[0].settings.theme); // "blue" - Original changed! 😱
```

8.2 📁 Array of Objects

```
let originalArray = [
  { id: 1, name: "Item 1" },
  { id: 2, name: "Item 2" }
];

// ❌ Wrong - shallow copy
let shallowCopy = [...originalArray];
shallowCopy[0].name = "Modified"; // Changes original! 😱

// ✅ Correct - deep copy
```

```
let deepCopy = originalArray.map(item => ({ ...item }));
deepCopy[0].name = "Modified"; // Doesn't change original ✨
```

8.3 🔄 Circular References

```
let obj = { name: "John" };
obj.self = obj; // Circular reference 🔄

// ✨ This will throw an error
// let copy = JSON.parse(JSON.stringify(obj)); // Error!

// 🛠 Custom solution needed
function deepCopyWithCircular(obj, visited = new WeakMap()) {
  if (obj === null || typeof obj !== "object") return obj;
  if (visited.has(obj)) return visited.get(obj);

  let copy = Array.isArray(obj) ? [] : {};
  visited.set(obj, copy);

  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      copy[key] = deepCopyWithCircular(obj[key], visited);
    }
  }

  return copy;
}
```

9. 🌐 Real-World Examples

9.1 🧠 React State Management

```
// 🛒 Shopping cart in React
const [cart, setCart] = useState([
  { id: 1, name: "Laptop", price: 999, quantity: 1 },
  { id: 2, name: "Mouse", price: 25, quantity: 2 }
]);

// ➕ Adding a new item (shallow copy sufficient)
const addItem = (newItem) => {
  setCart([...cart, newItem]);
};

// 🔄 Updating item quantity (need to copy the specific item)
const updateQuantity = (id, newQuantity) => {
  setCart(cart.map(item =>
```

```

    item.id === id
    ? { ...item, quantity: newQuantity } // Shallow copy of item
    : item
  ));
};

```

9.2 Form Data Handling

```

//  Complex form with nested data
const [formData, setFormData] = useState({
  personalInfo: {
    firstName: "",
    lastName: "",
    email: ""
  },
  address: {
    street: "",
    city: "",
    country: ""
  },
  preferences: {
    notifications: true,
    theme: "light"
  }
});

//  Updating nested form data
const updatePersonalInfo = (field, value) => {
  setFormData({
    ...formData,
    personalInfo: {
      ...formData.personalInfo,
      [field]: value
    }
  });
};

```

9.3 Redux State Management

```

//  Redux reducer
const userReducer = (state = initialState, action) => {
  switch (action.type) {
    case 'UPDATE_USER_PROFILE':
      return {
        ...state,
        users: state.users.map(user =>
          user.id === action.payload.id
            ? { ...user, ...action.payload.updates }
            : user
        )
      };
    default:
      return state;
  }
};

```

```

    )
  };

  case 'ADD_USER_HOBBY':
    return {
      ...state,
      users: state.users.map(user =>
        user.id === action.payload.userId
          ? {
              ...user,
              hobbies: [...user.hobbies, action.payload.hobby]
            }
          : user
        )
    };

  default:
    return state;
}
};

```

9.4 🌐 API Response Manipulation

```

// 📋 Processing API response without mutating original
const processApiResponse = (apiData) => {
  // 🕒 Deep copy to avoid mutating cached data
  const processedData = JSON.parse(JSON.stringify(apiData));

  processedData.users = processedData.users.map(user => ({
    ...user,
    fullName: `${user.firstName} ${user.lastName}`,
    isActive: user.status === 'active'
  }));

  return processedData;
};

```

10. ✨ Best Practices

10.1 🎯 Choose the Right Method

```

// 📋 For simple objects/arrays - use spread operator
const simpleObject = { a: 1, b: 2 };
const copy1 = { ...simpleObject };

// 🕒 For complex nested structures - use proper deep copy

```

```
const complexObject = { a: { b: { c: 1 } } };
const copy2 = structuredClone(complexObject); // Modern ✨
// or
const copy3 = JSON.parse(JSON.stringify(complexObject)); // Older browsers 🔄
```

10.2 ⚡ Performance Considerations

```
// 🔍 Shallow copy is faster - use when possible
const users = [...originalUsers]; // Fast ⚡

// 🕒 Deep copy is slower - use only when necessary
const users = originalUsers.map(user => ({ ...user })); // Better than full deep copy 🎯
```

10.3 🔒 Immutability Patterns

```
// ✅ Good: Creating new objects instead of mutating
const updateUser = (users, id, updates) => {
  return users.map(user =>
    user.id === id ? { ...user, ...updates } : user
  );
};

// ❌ Bad: Mutating original
const updateUserBad = (users, id, updates) => {
  const user = users.find(u => u.id === id);
  Object.assign(user, updates); // Mutates original! 🤖
  return users;
};
```

10.4 🛠️ Helper Functions

```
// 🎯 Create reusable utility functions
const copyArray = (arr) => [...arr];
const copyObject = (obj) => ({ ...obj });
const deepCopySimple = (obj) => JSON.parse(JSON.stringify(obj));

// 📁 For arrays of objects
const copyArrayOfObjects = (arr) => arr.map(item => ({ ...item }));

// 🎯 Usage
let usersCopy = copyArrayOfObjects(users);
```

11. 🎓 Advanced Concepts

11.1 🛠️ Custom Deep Copy with Type Handling

```
function advancedDeepCopy(obj, visited = new WeakMap()) {
  // 🧱 Primitive types
  if (obj === null || typeof obj !== "object") {
    return obj;
  }

  // 🔄 Check for circular reference
  if (visited.has(obj)) {
    return visited.get(obj);
  }

  // 🎯 Handle different object types
  if (obj instanceof Date) {
    return new Date(obj);
  }

  if (obj instanceof RegExp) {
    return new RegExp(obj);
  }

  if (obj instanceof Map) {
    const copy = new Map();
    visited.set(obj, copy);
    for (let [key, value] of obj) {
      copy.set(key, advancedDeepCopy(value, visited));
    }
    return copy;
  }

  if (obj instanceof Set) {
    const copy = new Set();
    visited.set(obj, copy);
    for (let value of obj) {
      copy.add(advancedDeepCopy(value, visited));
    }
    return copy;
  }

  // 📦 Handle Arrays
  if (Array.isArray(obj)) {
    const copy = [];
    visited.set(obj, copy);
    for (let i = 0; i < obj.length; i++) {
      copy[i] = advancedDeepCopy(obj[i], visited);
    }
    return copy;
  }

  // 📄 Handle Objects
  const copy = {};
```

```

visited.set(obj, copy);

for (let key in obj) {
  if (obj.hasOwnProperty(key)) {
    copy[key] = advancedDeepCopy(obj[key], visited);
  }
}

return copy;
}

```

11.2 Proxy-based Immutable Objects

```

function createImmutable(obj) {
  return new Proxy(obj, {
    set() {
      throw new Error("Cannot modify immutable object"); // 🚫
    },
    get(target, prop) {
      const value = target[prop];
      if (typeof value === 'object' && value !== null) {
        return createImmutable(value);
      }
      return value;
    }
  });
}

// 🎯 Usage
const immutableUser = createImmutable({ name: "John", age: 25 });
// immutableUser.name = "Jane"; // Throws error 💥

```

11.3 Copy-on-Write Pattern

```

class CopyOnWrite {
  constructor(data) {
    this._data = data;
    this._copied = false;
  }

  get data() {
    return this._data;
  }

  modify(updater) {
    if (!this._copied) {
      this._data = { ...this._data };
      this._copied = true;
    }
  }
}

```

```

    return updater(this._data);
  }
}

// 🎯 Usage
const cow = new CopyOnWrite({ name: "John", age: 25 });
cow.modify(data => data.age = 26); // Only copies when needed ⚡

```

12. 💪 Practice Exercises

📝 Exercise 1: Basic Copying

```

// 🎯 Create both shallow and deep copies of this object
const student = {
  name: "Alice",
  grades: [85, 92, 78],
  info: {
    age: 20,
    major: "Computer Science"
  }
};

// 🛠 Your solution here:
// 1. Create a shallow copy
// 2. Create a deep copy
// 3. Test by modifying nested properties

```

⚙️ Exercise 2: React State Update

```

// 🛠 Fix this React component to properly update nested state
const [state, setState] = useState({
  user: {
    name: "John",
    preferences: {
      theme: "dark",
      notifications: true
    }
  },
  posts: [
    { id: 1, title: "First Post", likes: 5 },
    { id: 2, title: "Second Post", likes: 3 }
  ]
});

// 🛠 Fix this function to properly update the theme
const updateTheme = (newTheme) => {

```



```
// Your solution here 🎯
};

// 🛠️ Fix this function to increment likes for a specific post
const incrementLikes = (postId) => {
  // Your solution here 🎯
};
```

🛠️ Exercise 3: Deep Copy Function

```
// 🎯 Create a deep copy function that handles:
// - Nested objects and arrays 📦
// - Dates 📅
// - null and undefined values ?
// - Circular references (bonus) 🔁

function myDeepCopy(obj) {
  // Your implementation here 💡
}

// 🧪 Test cases
const testObj = {
  str: "hello",
  num: 42,
  bool: true,
  date: new Date(),
  arr: [1, 2, { nested: "value" }],
  obj: { deep: { very: { deep: "value" } } }
};
```

⚡ Exercise 4: Performance Comparison

```
// 📊 Compare performance of different copying methods
const largeObject = {
  /* Create a large nested object */
};

// 📈 Measure and compare:
// 1. Spread operator (shallow) 🔍
// 2. JSON.parse(JSON.stringify()) (deep) 📋
// 3. Custom recursive function (deep) 🛠️
// 4. Lodash cloneDeep (if available) 📦

function benchmarkCopying() {
  // Your benchmarking code here 🎯
}
```

Summary

Key Takeaways:

1. 🧠 **Understanding is Critical:** Know when you're dealing with primitives vs references
2. 🎯 **Choose Wisely:** Use shallow copy when you only modify top-level properties, deep copy when you need complete independence
3. 🛠️ **Method Selection:** Spread operator for shallow, JSON methods for simple deep copy, custom functions for complex cases
4. ⚠️ **Watch for Pitfalls:** Be careful with nested objects, arrays of objects, and circular references
5. ⚡ **Performance Matters:** Shallow copy is faster, only use deep copy when necessary
6. 🌐 **Real-World Application:** Essential for React state management, Redux, and any data manipulation

Quick Reference:

- 🔍 **Shallow Copy:** `{...obj}` , `[...arr]` , `Object.assign()`
- 🕸️ **Deep Copy:** `JSON.parse(JSON.stringify())` , `structuredClone()` , custom recursive functions
- 📦 **Library Solutions:** `Lodash cloneDeep()` , `Ramda clone()`

Master these concepts, and you'll write more predictable, bug-free JavaScript code in your MERN stack applications! 🚀 ✨



About Me

 *FrontEnd Developer* •  *JavaScript Enthusiast* •  *React*



Let's Connect & Build Together!



LINKEDIN



FOLLOW ON X



GITHUB



"Building the future, one line of code at a time"



Tech Stack Mastery: JavaScript • React • Next.js • Tailwind CSS • GSAP •
Git & GitHub



Open for collaborations, code reviews, and tech discussions!



Let's create something amazing together!



Crafted with ❤️ for the JavaScript Community



****Keep Learning**** •  ****Keep Growing**** •  ****Keep Coding**** 

© 2025 Neeraj Kumar | Made with passion for developers worldwide 

